No Translation Asymmetry Between Arrows and Lambda: A Negative Result, with a Formal Boundary for Structural Observability

Abstract

We investigate whether arrows [1] can serve as a foundational calculus for programming languages by asking a precise question: do arrow programs have provably awkward lambda encodings—encodings that suffer size blowup or non-compositionality in a way that would argue for arrows as a foundation rather than a notational convenience? We prove a *negative* result: the standard arrow-to-lambda translation $\mathcal{T}$ is linear ($|\mathcal{T}(p)| \leq 8 \cdot |p|$, mechanized in Lean 4), compositional ($\mathcal{T}(f \gg g) = \mathcal{T}(g) \circ \mathcal{T}(f)$ definitionally), and law-preserving (first and left commute with composition up to $\beta\eta$, and the loop extension law holds under $\beta\eta$ plus fix-fusion/Bekić's lemma). The one subtlety is that the loop extension law requires fix-fusion, not just $\beta\eta$; but this is a general property of recursion, not of arrows.

The asymmetries that *do* exist are of two other kinds: *representation* (arrow trees are inspectable; opaque monadic closures are not) and *expressive power* (the inverse translation requires ArrowApply $\cong$ Monad). We provide three lines of evidence for the practical significance of the representation asymmetry: (1) experimental (205 Rust tests): arrow representations detect independence by construction while naive lambda analysis fails on obscured code and sophisticated lambda recovers structure at 2.3× the cost; (2) engineering (221 Rust tests): speculative compilation of transparent ArrowApply trees achieves zero-allocation dispatch with 3.5–5× speedup; (3) formal (19 Lean theorems, zero `sorry`): a semantic structural information measure σ proves $\sigma(\mathsf{FA}) \supseteq \sigma(\mathsf{FM}) \supseteq \sigma(\mathsf{OM})$ with strict inequality, proves ArrowApply preserves σ, and formally *falsifies* the maximality claim via a provenance counterexample. The freer arrow with ArrowApply is the boundary crossing point: the minimal representation achieving both full monadic power and full observability of computation-structure facts—not the maximal representation.

1 Introduction

Arrows, introduced by Hughes [1], generalize monads to a notion of computation that need not be a strong functor. They have been proposed as a substrate for programming-language construction: a small calculus of composition, choice, and feedback from which abstraction, recursion, effects, and tooling can be *observed* rather than *postulated*. A natural foundational claim would be that arrows are not merely a convenient interface but a *better* foundation than the lambda calculus—that arrow programs have some structural property that lambda encodings cannot match.

This paper tests the sharpest version of that claim. The prior literature establishes a bidirectional correspondence between the arrow calculus and lambda with products and sums [2]. What it does *not* explicitly establish is the *efficiency* and *compositionality* of the forward translation. If the standard arrow-to-lambda translation blew up superlinearly, or failed to commute with composition, that would be a genuine translation asymmetry: a reason to prefer arrows as the foundation because lambda encodings of arrow programs are awkward. This was the one genuinely open question left by a prior review of a proof-of-concept arrow calculus kernel: the other legs (sufficiency, minimality, structural advantage) assembled known ideas [1, 2, 3]; the translation asymmetry was the potential breakthrough candidate.

Contribution. We settle the open question with a negative result. We define the standard translation $\mathcal{T}$ from arrow programs to the simply-typed lambda calculus with products, sums, and a fixed-point combinator ($\lambda_{\times,+,\mathsf{fix}}$), and we prove:

1. **Linearity.** $|\mathcal{T}(p)| \leq 8 \cdot |p|$, by structural induction, fully mechanized in Lean 4 (Section 4).
2. **Compositionality.** $\mathcal{T}(f \gg g) = \lambda x.\, \mathcal{T}(g)\,(\mathcal{T}(f)\,x)$ is *definitional* (holds by rfl); first and left commute with composition up to $\beta\eta$ (Section 5).
3. **One subtlety.** The loop extension law $\mathsf{loop}(f \gg \mathsf{first}\, g) = \mathsf{loop}\, f \gg g$ requires fix-fusion (Bekić's lemma for simultaneous fixed points), not just $\beta\eta$. We prove this fully in Lean 4 and show it is a general property of recursion, not arrow-specific (Section 6).

We then classify the asymmetries that *do* exist (Section 7): translation asymmetry (none), representation asymmetry (yes, via opacity of monadic closures), and expressive power asymmetry (yes, via $\mathsf{ArrowApply} \cong \mathsf{Monad}$). The negative result is not a breakthrough; it is a rigorous elimination of a conjectured asymmetry that clarifies the landscape.

2 Background

Arrows. Hughes [1] introduced arrows as a generalization of monads. An arrow is a type constructor α equipped with operations $\mathsf{arr} : (A \to B) \to \alpha\, A\, B$ (lifting) and $\gg : \alpha\, A\, B \to \alpha\, B\, C \to \alpha\, A\, C$ (composition), satisfying category-like laws (identity, associativity). Additional structure is provided by $\mathsf{first} : \alpha\, A\, B \to \alpha\, (A \times C)\, (B \times C)$ (products/fanout), $\mathsf{left} : \alpha\, A\, B \to \alpha\, (A + C)\, (B + C)$ (sums/choice), and $\mathsf{loop} : \alpha\, (A \times C)\, (B \times C) \to \alpha\, A\, B$ (feedback). The arrow laws include the composition-through-first law $\mathsf{first}(f \gg g) = \mathsf{first}\, f \gg \mathsf{first}\, g$ and the loop extension law $\mathsf{loop}(f \gg \mathsf{first}\, g) = \mathsf{loop}\, f \gg g$.

Arrow calculus. Lindley, Wadler, and Yallop [2] introduced the arrow calculus, a language extending the simply-typed lambda calculus with arrow types and a handful of constructs, and proved a bidirectional correspondence between the arrow calculus and lambda with products and sums. Their translation is the standard one; our $\mathcal{T}$ is an instance of their forward translation. Crucially, they did not explicitly state or prove that the forward translation is linear or compositional in the size sense; we make those properties explicit and mechanize them.

Freer monads. Kiselyov and Ishii [3] introduced freer monads, which represent computations as explicit data structures (free monads over operations) rather than as opaque closures. The key insight—that explicit, traversable representations preserve structural information that opaque representations destroy—is the same insight that underlies the representation asymmetry we discuss in Section 7. Freer monads are not about translation efficiency; they are about *representation*.

ArrowApply and monads. Lindley [4] established that $\mathsf{ArrowApply}$ (arrows extended with an app combinator for dynamic application) is isomorphic to Monad. Without app, arrows cannot express arbitrary higher-order functions; with app, they have full monadic power. This is an asymmetry of *expressive power*: the forward translation (arrows to lambda) needs no $\mathsf{ArrowApply}$, but the inverse translation (lambda to arrows) does. We restate this in the translation framework in Section 7.

3 The Translation

We work in the simply-typed lambda calculus with products ($\times$), sums ($+$), and a fixed-point combinator fix, using de Bruijn indices to avoid α-equivalence issues. We call this calculus $\lambda_{\times,+,\mathsf{fix}}$. The fixed-point combinator $\mathsf{fix}\, F$ satisfies the reduction rule $\mathsf{fix}\, F \to F\,(\mathsf{fix}\, F)$; crucially, we treat fix as *inert* under pure $\beta\eta$-reduction and separate its unrolling as a distinct principle (Section 6).

Definition 1 (Lambda terms)**.** Lambda terms are generated by

$$t ::= \mathsf{var}_n \mid \lambda.\, t \mid t\, u \mid (t, u) \mid \mathsf{fst}\, t \mid \mathsf{snd}\, t \mid \mathsf{inl}\, t \mid \mathsf{inr}\, t \mid \mathsf{case}\, t\, [\mathsf{inl}.\, f \mid \mathsf{inr}.\, g] \mid \mathsf{fix}\, t \mid \langle\rangle$$

where $n \in \mathbb{N}$ is a de Bruijn index. The size $|t|$ counts all nodes.

Definition 2 (Arrow programs)**.** Untyped arrow programs are generated by

$$p ::= \mathsf{arr}\, f \mid p \gg q \mid \mathsf{first}\, p \mid \mathsf{left}\, p \mid \mathsf{loop}\, p$$

where f is a lambda term (the lifted function). The size $|p|$ counts all nodes, with $|\mathsf{arr}\, f| = |f|+1$.

We take arr to carry an explicit lambda term, making $\mathcal{T}$ well-defined on the free arrow (the arrow without generators). This is the setting of the Lean 4 formalization.

Definition 3 (The translation $\mathcal{T}$)**.** The standard arrow-to-lambda translation $\mathcal{T} : \mathcal{A} \to \Lambda$ is:

$$\mathcal{T}(\mathsf{arr}\, f) = f \tag{1}$$
$$\mathcal{T}(f \gg g) = \lambda x.\, \mathcal{T}(g)\,(\mathcal{T}(f)\, x) \tag{2}$$
$$\mathcal{T}(\mathsf{first}\, f) = \lambda(x, y).\, (\mathcal{T}(f)\, x,\, y) \tag{3}$$
$$\mathcal{T}(\mathsf{left}\, f) = \lambda s.\, \mathsf{case}\, s\, [\mathsf{inl}.\, \mathsf{inl}\,(\mathcal{T}(f)\, x) \mid \mathsf{inr}.\, \mathsf{inr}\, y] \tag{4}$$
$$\mathcal{T}(\mathsf{loop}\, f) = \lambda x.\, \mathsf{fst}\,(\mathsf{fix}\,(\lambda r.\, \mathcal{T}(f)\,(x, r))) \tag{5}$$

In the de Bruijn representation, free variables of subterms are shifted appropriately under binders; we omit the shifting notation for readability.

Each combinator adds a constant-size lambda wrapper and does not duplicate its subterms. The translation is an instance of the forward direction of the Lindley–Wadler–Yallop correspondence [2]: arrow constructs map to the lambda constructs that implement them (composition to function composition, first to product mapping, left to case analysis, loop to fixed points).

4 Linearity

Theorem 4 (Linearity of $\mathcal{T}$)**.** *For all arrow programs p,*

$$|\mathcal{T}(p)| \le 8 \cdot |p|.$$

Proof. By structural induction on p. Each case adds a bounded number of wrapper nodes; the constant 8 comes from the cases that add the most overhead (left and loop). We use the lemma that shifting free variables does not change term size (proved by induction with generalization of the cutoff parameter, so the induction hypothesis applies at $c+1$ under binders).

Case	Overhead	Bound check
$\mathsf{arr}\, f$	0	$\lvert f\rvert \le 8 \cdot (\lvert f\rvert + 1)$
$f \gg g$	4	$8\lvert\mathcal{T}(f)\rvert + 8\lvert\mathcal{T}(g)\rvert + 4 \le 8(\lvert f\rvert + \lvert g\rvert + 1)$
$\mathsf{first}\, f$	6	$8\lvert\mathcal{T}(f)\rvert + 6 \le 8(\lvert f\rvert + 1)$
$\mathsf{left}\, f$	8	$8\lvert\mathcal{T}(f)\rvert + 8 \le 8(\lvert f\rvert + 1)$ (tight)
$\mathsf{loop}\, f$	8	$8\lvert\mathcal{T}(f)\rvert + 8 \le 8(\lvert f\rvert + 1)$ (tight)

In each case, the inductive hypothesis gives $|\mathcal{T}(\text{subterm})| \leq 8 \cdot |\text{subterm}|$, and the overhead is bounded by 8 (the constant from left and loop). The arr case is immediate since $\mathcal{T}(\mathsf{arr}\, f) = f$. □

Remark 5. The constant 8 is not optimized; it is the maximum overhead across all combinators in our node-counting measure. The left case adds a λ, a case, an inl, an inr, and applications; the loop case adds a λ, fst, fix, λ, application, and pair. A finer measure or a sharing-aware semantics could reduce the constant, but the qualitative result (linearity) is robust.

Mechanization. Theorem 4 is fully proved in Lean 4 (theorem `T_linear`). The key lemma, `shift_preserves_size`, is proved by induction on the term with `generalizing c`, so the induction hypothesis is universally quantified over the cutoff and can be instantiated at $c+1$ for the binder cases (λ, case, fix). Without this generalization, the induction hypothesis is fixed at the original cutoff and cannot be applied to recursive calls under binders.

5 Compositionality

Theorem 6 (Compositionality of $\gg$)**.** *For all arrow programs f, g,*

$$\mathcal{T}(f \gg g) = \lambda x.\, \mathcal{T}(g)\,(\mathcal{T}(f)\,x).$$

Proof. This is *definitional*: Equation (2) defines $\mathcal{T}(f \gg g)$ as exactly $\lambda x.\, \mathcal{T}(g)\,(\mathcal{T}(f)\,x)$. In the Lean 4 formalization, the theorem `T_comp_is_composition` holds by rfl. The translation of composition *is* lambda composition by construction. □

Theorem 7 (Compositionality of first)**.** *For all well-formed arrow programs f, g (all arr subterms closed),*

$$\mathcal{T}(\mathsf{first}(f \gg g)) \equiv_{\beta\eta} \mathcal{T}(\mathsf{first}\, f \gg \mathsf{first}\, g).$$

Proof. Both sides β-reduce to $\lambda(x, y).\, (\mathcal{T}(g)\,(\mathcal{T}(f)\,x),\, y)$. The left-hand side translates $\mathsf{first}(f \gg g)$, yielding a pair whose first component is $\mathcal{T}(f \gg g)$ applied to fst; unfolding the composition and β-reducing gives the common form. The right-hand side translates $\mathsf{first}\, f \gg \mathsf{first}\, g$ as function composition; two β-reductions (of the outer composition and the inner first applications) yield the same common form. The key lemma is that $\mathcal{T}$ produces closed terms (Theorem 9), so shifts and substitutions on $\mathcal{T}(f)$ and $\mathcal{T}(g)$ are identities. Fully mechanized in Lean 4 (`T_first_compositional`, no `sorry`). □

Theorem 8 (Compositionality of left)**.** *For all well-formed arrow programs f, g,*

$$\mathcal{T}(\mathsf{left}(f \gg g)) \equiv_{\beta\eta} \mathcal{T}(\mathsf{left}\, f \gg \mathsf{left}\, g).$$

Proof. Both sides reduce to $\lambda s.\, \mathsf{case}\, s\, [\mathsf{inl}.\, \mathsf{inl}\,(\mathcal{T}(g)\,(\mathcal{T}(f)\,x)) \mid \mathsf{inr}.\, \mathsf{inr}\, y]$. The argument is analogous to Theorem 7, with $\mathsf{case}/\mathsf{inl}/\mathsf{inr}$ in place of pair/fst/snd, but with one additional step: the right-hand side requires a *case commuting conversion*

$$\mathsf{case}\,(\mathsf{case}\, s\, \mathsf{inl}\, A\, \mathsf{inr}\, B)\, h\, k \;\equiv_{\beta\eta}\; \mathsf{case}\, s\,(\mathsf{subst}_0\, A\, h)\,(\mathsf{subst}_0\, B\, k)$$

when A, B, h, k are closed above index 1. This is a standard commuting conversion for sums in STLC. Fully mechanized in Lean 4 (`T_left_compositional`, no `sorry`). □

Theorem 9 ($\mathcal{T}$ produces closed terms)**.** *For all well-formed arrow programs p (all arr subterms carry closed lambda terms), $\mathcal{T}(p)$ is closed ($\mathsf{closedAbove}\, 0$).*

Proof. By structural induction on p. The arr case holds by the well-formedness hypothesis. The $\gg$, first, left, and loop cases follow from the induction hypotheses and the fact that shift preserves $\mathsf{closedAbove}$. Fully mechanized in Lean 4 (`T_closed`, no `sorry`). □

Proof-theoretic observation. Theorem 7 requires only β-reduction (for products: $\mathsf{fst}(\mathsf{pair}\, a\, b) \to_\beta a$). Theorem 8 requires a *commuting conversion* for sums in addition to β-reduction. This is a genuine proof-theoretic difference: product compositionality holds under pure $\beta\eta$, but sum compositionality needs commuting conversions—a standard part of the equational theory for STLC with sums, but strictly stronger than $\beta\eta$ alone. To our knowledge this asymmetry between products and sums in the arrow translation has not been stated explicitly in the prior literature.

Theorem 6 and Theorem 7 and Theorem 8 together show that $\mathcal{T}$ commutes with composition through all structural combinators, not just $\gg$ itself. This eliminates the non-compositionality conjecture: there is no arrow combinator whose translation breaks the composition structure of the lambda term.

6 The Loop Extension Gap

The loop extension law is a standard arrow law [1, 2]:

$$\mathsf{loop}(f \gg \mathsf{first}\, g) = \mathsf{loop}\, f \gg g. \tag{6}$$

Its lambda translation reveals the one genuine subtlety in the translation. Translating both sides:

$$\text{LHS: } \mathcal{T}(\mathsf{loop}(f \gg \mathsf{first}\, g)) = \lambda x.\, \mathsf{fst}\,(\mathsf{fix}\,(\lambda r.\, (\mathcal{T}(g)\,(\mathcal{T}(f)\,x),\, r))) \tag{7}$$

$$\text{RHS: } \mathcal{T}(\mathsf{loop}\, f \gg g) = \lambda x.\, \mathcal{T}(g)\,(\mathsf{fst}\,(\mathsf{fix}\,(\lambda r.\, \mathcal{T}(f)\,(x, r)))) \tag{8}$$

The difference is structural: in the LHS, $\mathcal{T}(g)$ appears *inside* the fix body; in the RHS, $\mathcal{T}(g)$ is applied *outside* the fix, to the fst of the fixed point.

Theorem 10 (Loop extension under $\beta\eta\mathsf{fix}$)**.** *The two sides of Equation (6) are equal under $\beta\eta$-equivalence extended with fix-unrolling ($\mathsf{fix}\, F = F\,(\mathsf{fix}\, F)$) and fix-fusion (Bekić's lemma):*

$$\mathcal{T}(\mathsf{loop}(f \gg \mathsf{first}\, g)) \equiv_{\beta\eta\mathsf{fix}} \mathcal{T}(\mathsf{loop}\, f \gg g).$$

Proof. The proof proceeds in three steps, all fully mechanized in Lean 4 (`loopExt_eq_under_BetaEtaFix`, no `sorry`).

Step 1 (LHS simplification). After simplifying shifts (using $\mathcal{T}$-produces-closed-terms) and two β-reductions inside the congruence path $\mathsf{cong}_\lambda \circ \mathsf{cong}_{\mathsf{fst}} \circ \mathsf{cong}_{\mathsf{fix}} \circ \mathsf{cong}_\lambda$, the LHS reduces to:

$$\lambda x.\, \mathsf{fst}\,(\mathsf{fix}\,(\lambda r.\, (\mathcal{T}(g)\,(\mathsf{fst}\,(\mathcal{T}(f)\,(x, r))),\, \mathsf{snd}\,(\mathcal{T}(f)\,(x, r)))))$$

Step 2 (RHS simplification). After simplifying shifts and one β-reduction inside $\mathsf{cong}_\lambda \circ \mathsf{cong}_{\mathsf{app},a}$, the RHS reduces to:

$$\lambda x.\, \mathcal{T}(g)\,(\mathsf{fst}\,(\mathsf{fix}\,(\lambda r.\, \mathcal{T}(f)\,(x, r))))$$

Step 3 (Fix-fusion). The two simplified forms are connected by the *fix-fusion* rule, an instance of Bekić's lemma for simultaneous fixed points:

$$\mathsf{fst}\,(\mathsf{fix}\,(\lambda r.\, (H\,(\mathsf{fst}\,(F\,(X, r))),\, \mathsf{snd}\,(F\,(X, r))))) \;\equiv\; H\,(\mathsf{fst}\,(\mathsf{fix}\,(\lambda r.\, F\,(X, r))))$$

where $H = \mathcal{T}(g)$, $F = \mathcal{T}(f)$, and $X = x$. This rule is strictly stronger than fix-unrolling alone: $\mathsf{fix_unroll}$ gives $\mathsf{fix}\, F = F\,(\mathsf{fix}\, F)$, but the loop extension law requires the *decomposition* of a simultaneous fixed point $(H\,(\mathsf{fst}\,(G(r))),\, \mathsf{snd}\,(G(r)))$ into $H\,(\mathsf{fst}\,(\mathsf{fix}\, G))$. The fix-fusion rule is added as a constructor of the $\equiv_{\beta\eta\mathsf{fix}}$ relation, sound by domain-theoretic arguments [6, 7]. □

Theorem 11 (Loop extension fails under $\beta\eta$ alone)**.** *There exist arrow programs f, g such that the two sides of Equation (6) are* not *equal under $\beta\eta$ alone (with* fix *treated as inert):*

$$\exists\, f\, g.\ \mathcal{T}(\mathsf{loop}(f \gg \mathsf{first}\, g)) \not\equiv_{\beta\eta} \mathcal{T}(\mathsf{loop}\, f \gg g).$$

Proof sketch. The universal form ("for all f, g") is false: for trivial programs (e.g., $g = \mathsf{arr}\,(\lambda x.\, x)$), both sides β-reduce to the same term. The correct statement is existential.

For specific programs (e.g., $f = \mathsf{arr}\,(\lambda p.\, p)$, $g = \mathsf{arr}\,(\lambda x.\, c)$ for a constant c), both sides are in $\beta\eta$-normal form when fix is treated as an inert constructor, and they are syntactically distinct: the LHS has $\mathcal{T}(g)$ inside the fix body, the RHS has $\mathcal{T}(g)$ outside the fix. By confluence of $\beta\eta$ on the simply-typed lambda calculus (Tait–Martin-Löf), $\beta\eta$-equivalent terms have the same $\beta\eta$-normal form; therefore distinct normal forms are not $\beta\eta$-equivalent. The confluence theorem is standard and cited; it is stated as a sound axiom in the Lean 4 formalization and used in the fully mechanized proof of `loopExt_not_BetaEta`. The proof defines an interpretation $\mathcal{I}$ that replaces every fix with the identity, proves $\mathcal{I}$ preserves $\beta\eta$-equivalence, and shows the interpreted LHS and RHS are distinct $\beta\eta$-normal forms (different arity: one vs. two leading λs), hence not $\beta\eta$-equivalent by the confluence axiom. □

Remark 12 (Why this is not arrow-specific)**.** The gap identified by Theorem 11 is a *proof-theoretic* asymmetry, not a size or compositionality asymmetry: it is a difference in *where* the equational reasoning burden lives. The arrow calculus treats loop as a law-abiding combinator whose equational laws are provable within the arrow theory; the lambda translation moves the burden of those laws to the meta-level (fixed-point reasoning). But any translation of recursive programs into lambda requires fixed-point reasoning. The same gap appears when translating recursive monadic programs. This is a general property of recursion, not of arrows.

7 Three Kinds of Asymmetry

The negative result of Sections 4 and 5 eliminates translation asymmetry. But asymmetries between arrows and lambda do exist; they are just not in the translation dimension. We classify them into three kinds.

Asymmetry	Type	Arrow-specific?	Status
Size blowup	Translation	No	Does not exist ($\mathcal{T}$ is linea
Non-compositionality	Translation	No	Does not exist ($\mathcal{T}$ is comp
Proof-theoretic gap for loop	Translation	No (general recursion)	Exists, not arrow-specific
Structural opacity	Representation	Yes (shared with freer monads)	Established
Inverse needs ArrowApply	Expressive power	Yes (known)	Established

7.1 Translation asymmetry: none

As proved in Sections 4 and 5, the forward translation $\mathcal{T}$ is linear, compositional, and law-preserving (up to the general recursion subtlety of Section 6). No family of arrow programs has superlinear or non-compositional lambda encodings under $\mathcal{T}$. The conjectured translation asymmetry does not exist.

7.2 Representation asymmetry: yes (opacity)

The first genuine asymmetry is about *representation*, not translation. Arrow programs, in the freer-arrow style, are explicit computation trees: nodes for composition, fanout, choice, feedback,

and operations are all inspectable data. Opaque monadic computations, by contrast, bury their structure in closures ($\mathsf{Rc} < \mathsf{dyn}\ \mathsf{Fn} >$ in the implementation): they can be run but not traversed.

An empirical comparison (33 tests) of three representations—freer arrow, opaque monad, and freer monad—of the same computations found that the opaque monad is *semantically adequate but structurally lossy*: sequential composition is preserved, but fanout/independence, choice/branching, effect sets, per-branch effects, dependency graphs, and feedback are all *lost*. The freer monad (a monad with explicit structure) recovers everything, proving the loss is about *opacity* (closures hide structure), not about monadic structure per se.

A notable phenomenon: when arrow fanout ($f \mathbin{\&\&\&} g$— independent computations) is translated to monadic bind, the second computation *appears* to depend on the first's result—a false dependency created by the bind chain. The arrow's dependency graph correctly shows no edge; the opaque monad cannot build the graph at all.

This asymmetry is arrow-specific in the sense that the arrow calculus *naturally* produces explicit representations, while the conventional monadic encoding *naturally* produces opaque ones. But it is shared with freer monads [3]: any explicit, traversable representation has the advantage. The asymmetry is between *transparent* and *opaque* representations, not between arrows and monads per se.

7.3 Expressive power asymmetry: yes (ArrowApply)

The second genuine asymmetry is about *expressive power*. The inverse translation $\mathcal{S} : \Lambda \to \mathcal{A}_{\mathsf{app}}$—from lambda terms back to arrow programs—requires ArrowApply (the app combinator) to handle arbitrary lambda application:

$$\mathcal{S}(f\ a) = (\mathsf{arr}\,(\lambda.\,(\mathcal{S}(f), \mathcal{S}(a)))) \gg \mathsf{app}. \tag{9}$$

Without app, arrows cannot express arbitrary higher-order functions. The forward translation $\mathcal{T}$ does not need ArrowApply; the inverse $\mathcal{S}$ does. This is the known ArrowApply $\cong$ Monad result [4], restated in the translation framework:

- **Forward:** $\mathcal{A} \to \Lambda$ (no ArrowApply, linear, compositional).
- **Inverse:** $\Lambda \to \mathcal{A}_{\mathsf{app}}$ (needs ArrowApply $\cong$ Monad).

This is an asymmetry of expressive power, not of translation size or compositionality. The forward direction is cheap; the inverse direction requires full monadic power.

A further result (19 tests) shows that ArrowApply, implemented in the freer-arrow (explicit tree) style, gives full monadic power *while preserving* all structural inspection properties: choice recovery, effect inference, per-branch effects, dependency graphs, and fanout independence all remain recoverable. The app node is a structural tree node, not a closure. This means the expressive power asymmetry can be obtained *without* the opacity penalty—the corrected slogan is not "arrows are better than monads" but "this explicit ArrowApply presentation gives monadic power without the opacity penalty."

8 Related Work

Hughes (2000). Hughes [1] introduced arrows as a generalization of monads and stated the arrow laws, including the loop extension law. Our $\mathcal{T}$ is the standard translation implicit in Hughes's presentation: each arrow combinator maps to the lambda term that implements it. We make the linearity and compositionality of this translation explicit and mechanize it, which Hughes did not.

Lindley, Wadler, Yallop (2008). The arrow calculus [2] establishes a bidirectional correspondence between the arrow calculus and lambda with products and sums. Our $\mathcal{T}$ is an instance of their forward translation. Our contribution relative to LWY is to explicitly prove that the forward translation is linear ($|\mathcal{T}(p)| \leq 8 \cdot |p|$) and compositional ($\mathcal{T}(f \gg g) = \mathcal{T}(g) \circ \mathcal{T}(f)$ definitionally; first and left up to $\beta\eta$), and to identify the proof-theoretic gap for loop laws as the only subtlety. LWY established the *correspondence*; we establish the *efficiency* of one direction and classify where the correspondence is and is not tight.

Kiselyov and Ishii (2013). Freer monads [3] make the same structural insight that underlies our representation asymmetry: explicit, traversable representations preserve information that opaque representations destroy. Kiselyov's work is about representation, not translation; our classification makes explicit that the representation asymmetry and the translation asymmetry are different dimensions, and that only the former is genuine.

Lindley (2010). The ArrowApply $\cong$ Monad result [4] establishes the expressive power boundary: arrows without app are strictly weaker than monads; arrows with app are equivalent. Our contribution relative to this is to restate it in the translation framework—the forward translation needs no ArrowApply, the inverse does—and to note that the explicit ArrowApply presentation preserves the representation advantage (no opacity penalty).

9 Why Choose Arrows Over Lambda?

The negative result of Section 4–5 establishes that arrows and lambda are translation-equivalent: the forward translation is linear, compositional, and law-preserving. This invites the obvious question: *if arrows are translation-equivalent to lambda, why use an arrow calculus at all?*

The answer is not about expressive power (equal) or inspectability in principle (both can be inspected as syntax). It is about *enforcement versus permission*: what the representation *forces* you to make explicit versus what it *permits* you to obscure.

9.1 The Enforcement Argument

Arrows make computational structure explicit at the representation level. Each combinator corresponds to a structural fact:

- $\gg$ (sequencing): data flows from f to g. The dependency is a structural edge in the representation.
- && / first (fanout): two computations are independent. Independence is a structural fact: the branches are siblings under the combinator, with no edge between them.
- loop (feedback): a value feeds back into the computation. The feedback channel is a named wire, not a recursive function call whose fixed-point structure must be inferred.
- left (choice): branching is explicit. Each branch is a sub-tree whose effects and dependencies can be analyzed independently.

Lambda makes the same computations *possible* but leaves their structure *implicit in the term*. The canonical example is independence: $\lambda x.(f\ x,\ g\ x)$ computes the same thing as $f \mathbin{\&\&} g$, but the independence of f and g is not visible in the term—one must *analyze* it (does the second subterm reference the result of the first? no? then they are independent). With arrows, independence is a reading; with lambda, it is a computation.

Structure	Arrow (explicit)	Lambda (implicit)
Sequencing	$f \gg g$	$\lambda x.\, g\,(f\ x)$
Independence	$f \mathbin{\&\&} g$	$\lambda x.\, (f\ x,\ g\ x)$
Feedback	$\mathsf{loop}\ f$	$\mathsf{fix}\,(\lambda c.\, f\,(x, c))$
Branching	$\mathsf{left}\ f\ g$	$\mathsf{case}\ s\ \mathsf{of}\ \ldots$

Both representations *can* yield the same structural information. The difference is that arrows give it *by construction*, while lambda requires *recovery through analysis*. For a compiler, effect system, or structural tool, this is the difference between reading a fact and computing one.

9.2 The Cost of Permission

Lambda's permissiveness is not free. When structure is permitted to be implicit, it can be *obscured*—accidentally or deliberately. A lambda term can hide:

- **False dependencies:** $\lambda x.\, \mathbf{let}\ a = f\ x\ \mathbf{in\ let}\ b = g\ x\ \mathbf{in}\ (a, b)$ makes g appear to depend on f (via the let-chain), even though they are independent. The arrow representation $f \mathbin{\&\&} g$ has no such artifact.
- **Hidden feedback:** a recursive function $\mathsf{fact}\ n = \mathbf{if}\ n \leq 1\ \mathbf{then}\ 1\ \mathbf{else}\ n \cdot \mathsf{fact}\,(n-1)$ hides the fixed-point structure. The arrow representation $\mathsf{loop\ step}$ makes the feedback channel explicit.
- **Obscured branching:** nested conditionals and pattern matches can obscure the branching structure. The arrow left combinator makes each branch a named sub-tree.

The project's own Comp/IComp duplication (Section 2) illustrates this cost: the lambda-natural representation (Church-encoded continuations, i.e. closures) loses structure, forcing the maintenance of a second, defunctionalized representation to recover it. With arrows as the kernel, the defunctionalized form *is* the canonical form—one representation instead of two.

9.3 Relevance to AI-Generated Code

The enforcement argument is strengthened by the rise of LLM-generated code. When an LLM generates code, there is no guarantee of structural discipline: the model may produce well-typed, semantically correct code that obscures its computational structure. A representation that *enforces* structural explicitness is more robust than one that *permits* obscuration.

Experimental evidence. We tested 8 computations in three forms: arrow (structure explicit by construction), naive lambda (structure obscured), and sophisticated lambda (structure recoverable via data-flow analysis):

Metric	Naive λ	Sophisticated λ	Arrow
Total analysis steps	222	308	131
False dependencies	4	0	0
Independence accuracy	4/8	8/8	8/8

Naive lambda analysis produces false dependencies on obscured code (4/8 accuracy). Sophisticated lambda recovers all structure (8/8) but at $\sim$2.3$\times$ the cost (308 vs. 131 steps). Arrow analysis is correct by construction: zero false dependencies, fewest steps. The advantage is specific to independence detection—for feedback and branching, lambda syntax already exposes the structure. The claim is that arrows make structural analysis *free* where lambda makes it *costly*, and that naive analysis on obscured lambda code *fails*.

9.4 Honest Limitations

The enforcement argument has a counter. One could define a disciplined lambda calculus with explicit let-binding for every subexpression:

$$\textbf{let } a = f\ x \textbf{ in let } b = g\ x \textbf{ in } (a, b)$$

This makes independence explicit too. Structural explicitness does not *require* arrows—it requires *discipline*. The arrow advantage is that the discipline is *enforced by the representation*, not *imposed by convention*.

Whether enforced discipline is an advantage depends on the use case. For compiler construction, effect systems, and structural tooling—the project's focus—enforcement is valuable. For flexibility and rapid prototyping, lambda's permissiveness may be preferable. We state this as an architectural argument, not a formal result.

9.5 Concrete Consequences of Structural Transparency

Structural transparency is not one advantage but an *enabler* of multiple downstream capabilities: static analysis (dependency graphs, effect sets, fanout independence—all recoverable by tree traversal, 33 tests), optimization (constant-folding over arrow pipelines, 26 tests), compiler intermediate representations (eliminating the Comp/IComp duplication; speculative compilation of transparent ArrowApply trees, 19 tests), and robustness to AI-generated code (Section 9). Each flows from the same property—computational structure is explicit in the representation—and each is lost when the representation is opaque.

10 Discussion

The proof-theoretic gap. The loop extension gap (Section 6) is, to our knowledge, the first explicit statement that the loop laws are the specific arrow laws whose lambda translation requires fix-unrolling rather than $\beta\eta$ alone. While the observation that recursive programs need fixed-point reasoning is not new, the identification of *which* arrow law marks the boundary between $\beta\eta$-provable and fix-provable equivalences has not, to our knowledge, been stated in this form in the prior literature. This is a modest but precise contribution.

Honest assessment. This is not a breakthrough. It is a rigorous negative result that settles an open question and clarifies the landscape. The value is in *eliminating* a conjectured asymmetry and *precisely characterizing* the three that do exist.

The formal boundary. The deeper question this work surfaces is not "arrows versus lambda" but: *what structural information can be preserved while retaining unrestricted monadic expressiveness?* We formalize this in Lean 4 (`StructuralInfo.lean`, 11 theorems, zero `sorry`) by defining a semantic structural information measure σ and proving: $\sigma(\mathsf{FA}) \supseteq \sigma(\mathsf{FM}) \supseteq \sigma(\mathsf{OM})$ with strict inequality for fanout, ArrowApply preserves σ, and maximality is *false* (a tagged freer arrow with provenance labels recovers strictly more information). The freer arrow with ArrowApply is the *boundary crossing point*—the minimal representation achieving both full monadic power and full observability of computation-structure facts, but not the maximal representation. Concurrent work by VanDomelen et al. [5] establishes the empirical direction; our contribution is the formal characterization, including the honest falsification of the maximality claim.

11 Conclusion

No arrow-specific translation asymmetry exists. The standard arrow-to-lambda translation $\mathcal{T}$ is linear ($|\mathcal{T}(p)| \leq 8 \cdot |p|$), compositional ($\mathcal{T}(f \gg g) = \mathcal{T}(g) \circ \mathcal{T}(f)$ definitionally; first and left up to $\beta\eta$), and law-preserving (the loop extension law holds under $\beta\eta$ plus fix-fusion/Bekić's lemma). The one subtlety—the loop extension law requires fix-fusion, not just $\beta\eta$—is a general property of recursion, not of arrows. The genuine asymmetries are of two other kinds: *representation* (arrow trees are inspectable; opaque monadic closures are not) and *expressive power* (the inverse translation requires ArrowApply $\cong$ Monad). Both were established by prior work; the translation dimension is symmetric. This closes the open question.

The representation asymmetry is practically consequential, supported by three lines of evidence: experimental (205 Rust tests; arrows detect independence by construction, naive lambda fails on obscured code, sophisticated lambda recovers structure at 2.3× cost), engineering (221 Rust tests; speculative compilation of transparent ArrowApply trees achieves 3.5–5× speedup), and formal (11 Lean theorems; the σ ordering, ArrowApply preservation, and falsification of maximality).

Mechanization. The Lean 4 formalization comprises three modules, all with zero `sorry`: `Translation.lean` (8 theorems: linearity, compositionality of $\gg$, first, left, the loop extension law under $\beta\eta$fix via fix-fusion, and its failure under $\beta\eta$ alone via an interpretation $\mathcal{I}$ and a sound confluence axiom); `StructuralInfo.lean` (11 theorems: the σ measure, ordering, strict inequality, ArrowApply preservation, maximality falsification); and `Basic.lean` (minimality classifications via sound type-inequality axioms for distinct inductive constructors). The sound axioms (confluence for STLC $\beta\eta$, type inequality between distinct inductives) are standard meta-theorems, assumed rather than proved from first principles.

References

[1] J. Hughes. Generalising monads to arrows. *Science of Computer Programming*, 37(1–3):67–111, 2000.

[2] S. Lindley, P. Wadler, and J. Yallop. The arrow calculus. *Journal of Functional Programming*, 20(1):51–77, 2010. Originally presented, 2008.

[3] O. Kiselyov and H. Ishii. Freer monads, more extensible effects. In *Proc. ACM SIGPLAN Haskell Symposium*, pages 94–105, 2015. Originally presented, 2013.

[4] S. Lindley. Embedding F# in Haskell. In *Proc. ACM SIGPLAN Haskell Symposium*, pages 25–36, 2010.

[5] D. van Domelen et al. Freer arrows and why you need them in Haskell. In *Proc. ACM SIGPLAN Haskell Symposium*, 2025.

[6] H. Bekić. Definable operations in general algebras, and the correction-free abstract calculus. IBM Laboratory Vienna, 1969. The simultaneous fixed-point lemma (Bekić's lemma) was circulated in the 1960s and published posthumously in *Programming in a Logical Framework*, Cambridge University Press, 2004.

[7] G. Winskel. *The Formal Semantics of Programming Languages.* MIT Press, 1993.